Food Store: modelo documental

Integrantes: Tomas Martin, Benjamin Lagos, Santiago Prandina, Emir Gris, Leandro Rea, Brian Rampone, Joel Herrera, Octa Medina.

Fecha: 06/11/2026 · **Motores:** PostgreSQL 17.11 y MongoDB 8.0.10.
Origen: foodstorecopia · **Destino:** foodstorecopia_nosql.pedidos_u7.
github.com/Tomimartin/BaseDeDatos2

1. Subconjunto y claves

Se migraron los pedidos **180991, 181008, 181019, 181036 y 181045**, con una línea cada uno. pedido se vincula con detalle_pedido por id_pedido, con cliente por id_cliente y con producto a través de id_producto. Se conservaron los identificadores originales.

Campo	Tipo	Correspondencia
pedido_id	Long	Conserva pedido.id_pedido. Índice único.
fecha / fecha_origen_pg	Date / string	Fecha UTC consultable y valor PostgreSQL completo.
estado / eliminado	string / boolean	ACTIVO=false; ELIMINADO=true, por baja lógica.
forma_pago	string	Valor original del enum relacional.
cliente	objeto embebido	ID original, alias público y marca de anonimización.
items	arreglo de objetos	ID/nombre de producto, cantidad y precio histórico.
precio_unitario / subtotal / total	Decimal128	Importes decimales; total = suma de líneas.

Cliente e ítems se embeben porque se recuperan junto al pedido completo. Los nombres públicos de cliente son alias derivados del ID; los precios históricos provienen del detalle. [1, 3]

Estado del modelo real: vigencia por baja lógica. ACTIVO corresponde a eliminado=false y ELIMINADO a eliminado=true. Se mantiene el booleano de origen y se aplica el mismo CASE en PostgreSQL.

2. Ejemplo de documento

```
{
  pedido_id: NumberLong("180991"),
  fecha: ISODate("2026-01-31T04:19:57.926Z"),
  fecha_origen_pg: "2026-01-31 01:19:57.926064-03",
  estado: "ELIMINADO", eliminado: true, forma_pago: "TARJETA",
  cliente: { cliente_id: NumberLong("18492"),
    nombre_publico: "Cliente 18492", anonimizado: true },
  items: [{ producto_id: NumberLong("30998"),
    nombre_producto: "Producto generado 30998", cantidad: NumberInt(11),
    precio_unitario: NumberDecimal("1350.33"),
    subtotal: NumberDecimal("14853.63") }],
  total: NumberDecimal("14853.63")
}
```

Ejemplo reproducido del script ejecutado y del CSV validado. BSON Date conserva milisegundos; fecha_origen_pg retiene el valor completo con microsegundos y zona horaria. Los importes se guardan como Decimal128. [5, 6]

Migración y consultas ejecutadas

3. Carga e integridad

El CSV exportado desde PostgreSQL se conserva sin modificar. La versión entregada de consultas_mongodb.js se ejecutó pegando el contenido en la consola integrada, que no admitía load(). Crea pedidos_u7, aplica un índice único y reemplaza o incorpora los cinco IDs mediante bulkWrite/replaceOne con upsert.

Reglas comprobadas: cantidades positivas; precios no negativos; total igual a la suma de sus líneas; ID de pedido único y estado coherente con eliminado. La colección se creó con validación de tipos y valores.

4. Las cuatro consultas MongoDB

```
const pedidos = db.getSiblingDB("foodstorecopia_nosql")
  .getCollection("pedidos_u7");
const ids = [180991, 181008, 181019, 181036, 181045]
  .map(id => NumberLong(String(id)));
const alcance = { pedido_id: { $in: ids } };

// a) Todos, con identificador y estado.
pedidos.find(alcance, { _id: 0, pedido_id: 1, estado: 1 });
// b) Estado existente en los documentos.
pedidos.find({ ...alcance, estado: "ACTIVO" });
// c) Producto dentro del arreglo de lineas.
pedidos.find({ ...alcance, "items.producto_id": NumberLong("30998") });
// d) Total superior al umbral elegido.
pedidos.find({ ...alcance, total: { $gt: NumberDecimal("10000.00") } });
```

El umbral 10000 distingue al pedido de 7.120,15 de los otros cuatro. Es una selección del laboratorio.

Consulta	Cantidad	IDs observados
a) Todos	5	180991, 181008, 181019, 181036, 181045
b) Estado ACTIVO	3	181019, 181036, 181045
c) Producto 30998	1	180991
d) Total > 10000	4	180991, 181008, 181019, 181036

5. Evidencia de ejecución

Resultado: EXITO · **MongoDB:** 8.0.10 · **Hora local:** 06/11/2026 16:44:09 (-03:00).
Integridad: 0 pedidos con ítems inválidos y 0 totales inconsistentes.

```
4a todos: cantidad=5; IDs=180991, 181008, 181019, 181036, 181045
4b estado ACTIVO: cantidad=3; IDs=181019, 181036, 181045
4c producto 30998: cantidad=1; IDs=180991
4d total mayor a 10000: cantidad=4; IDs=180991, 181008, 181019, 181036
EXITO: migracion y cuatro consultas verificadas contra el CSV de PostgreSQL
```

Extracto de la salida de consola aportada por el equipo. El registro original usa UTC (19:44:09Z); se conservan la transcripción y el objeto resultado en evidencias/.

Comparación verificable entre motores

6. Mismo subconjunto, cantidades e identificadores

Las consultas de PostgreSQL utilizan los mismos cinco IDs. El estado se obtiene con CASE desde eliminado; el producto se verifica con EXISTS en detalle_pedido. La búsqueda por producto incluye activos y eliminados en ambos motores.

Consulta	Cant. PG	IDs PostgreSQL	Cant. Mongo	IDs MongoDB	Coinciden
Estado ACTIVO	3	181019, 181036, 181045	3	181019, 181036, 181045	Sí
Producto 30998	1	180991	1	180991	Sí

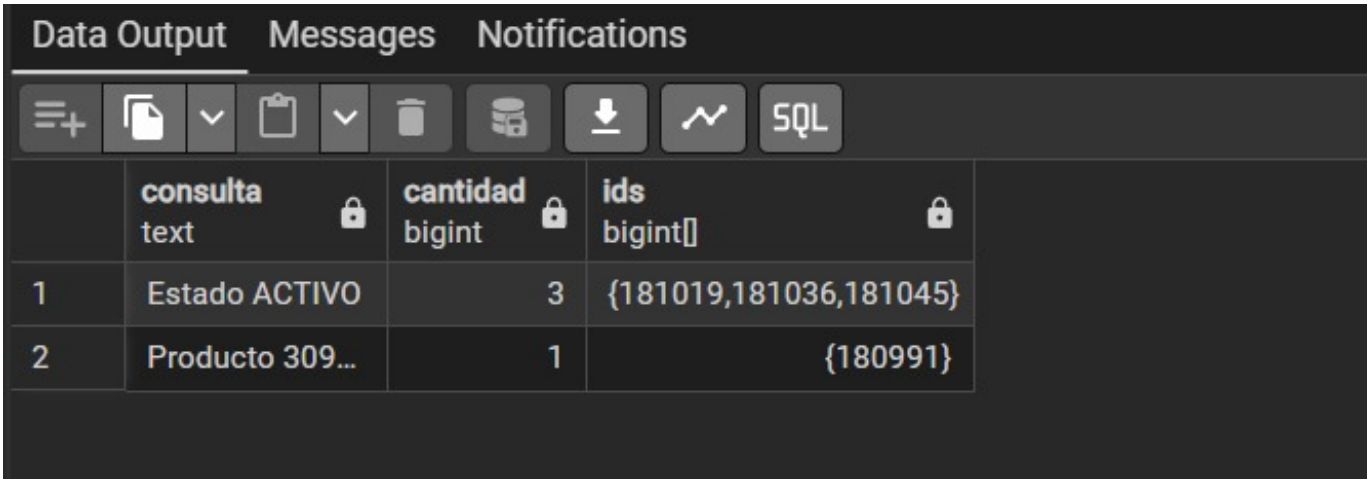
La captura de ejecución confirma las dos filas. El producto 30998 corresponde solo al pedido 180991 en este subconjunto; el pedido 181019 contiene realmente el producto 31026.

7. Datos del origen conservados

Pedido	Estado	Cliente ID	Producto ID	Cant.	Precio histórico	Total
180991	ELIMINADO	18492	30998	11	1.350,33	14.853,63
181008	ELIMINADO	2412	31015	8	4.384,97	35.079,76
181019	ACTIVO	4790	31026	19	3.544,59	67.347,21
181036	ACTIVO	13628	31043	16	3.324,05	53.184,80
181045	ACTIVO	4026	31052	5	1.424,03	7.120,15

Cada pedido tiene una línea en el CSV. Los totales se obtuvieron con cantidades y precios históricos exactos, y la ejecución verificó los documentos leídos de MongoDB contra esos datos.

8. Evidencia PostgreSQL



Recorte de la captura original, conservada completa como 03_comparacion_postgresql.png. La transcripción de sus dos filas está identificada como tal en evidencias/.

Evaluación de tecnología y asistencia de IA

9. Consulta guía y respuesta de IA

Herramienta: Codex. Consulta guía aplicada en esta revisión: Compará PostgreSQL y MongoDB para Food Store. El modelo relacional tiene pedido, detalle_pedido, producto y cliente; se consultan pedidos completos con frecuencia y se deben conservar cantidades y precios históricos. Evaluá embeber cliente e ítems, la atomicidad, la coherencia del stock y el costo de mantener dos motores. Diferenciá ventajas de diseño de afirmaciones de rendimiento que requieren mediciones.

Síntesis de respuesta: PostgreSQL resulta adecuado como fuente transaccional del negocio. MongoDB permite agrupar cliente e ítems para leer pedidos completos. Una copia del precio histórico conserva el valor migrado, pero necesita reglas de actualización. La ventaja de rendimiento debe comprobarse con una carga comparable.

10. Contraste de afirmaciones del informe anterior

Afirmación	Verificación técnica	Fuentes
Incrustar el precio garantiza su inmutabilidad.	Corregida: la copia histórica es independiente del catálogo, pero puede actualizarse. En el laboratorio se comprobó la igualdad de precios con PostgreSQL; no se implementó una prohibición general de cambios.	[2, 3]
PostgreSQL necesariamente tiene más latencia por usar JOINs.	No demostrada: embeber permite una operación de base de datos, sin garantizar una única lectura física ni menor tiempo. No se midieron latencias comparables en este ensayo.	[1, 4]

Las escrituras de un documento son atómicas. Eso no vuelve atómicas las cinco operaciones del lote ni sincroniza por sí mismo un catálogo separado. [2]

11. Conclusión técnica

Para Food Store conviene mantener PostgreSQL como fuente transaccional principal: sus relaciones y restricciones permiten coordinar clientes, pedidos, detalles y stock. MongoDB es adecuado para una representación de lectura de pedidos completos, con cliente e ítems embebidos y una actualización atómica por documento. Este laboratorio verificó correspondencia de datos y resultados para cinco pedidos, pero no midió una ventaja de rendimiento. Combinar motores agregaría trabajo de sincronización y control de consistencia. Por eso, una proyección MongoDB debería adoptarse cuando exista una necesidad de lectura concreta y una mejora medida que justifique ese costo. Los precios históricos deben conservarse desde detalle_pedido y protegerse mediante reglas de actualización, cualquiera sea el motor elegido.

Referencias oficiales

[1] Datos embebidos en MongoDB. Documentación oficial consultada el 06/11/2026.

[2] Atomicidad y transacciones. Documentación oficial consultada el 06/11/2026.

[3] Datos duplicados e históricos. Documentación oficial consultada el 06/11/2026.

[4] PostgreSQL: planes y EXPLAIN. Documentación oficial consultada el 06/11/2026.

[5] Tipos de datos de mongosh. Documentación oficial consultada el 06/11/2026.

[6] Extended JSON y precisión de fechas. Documentación oficial consultada el 06/11/2026.